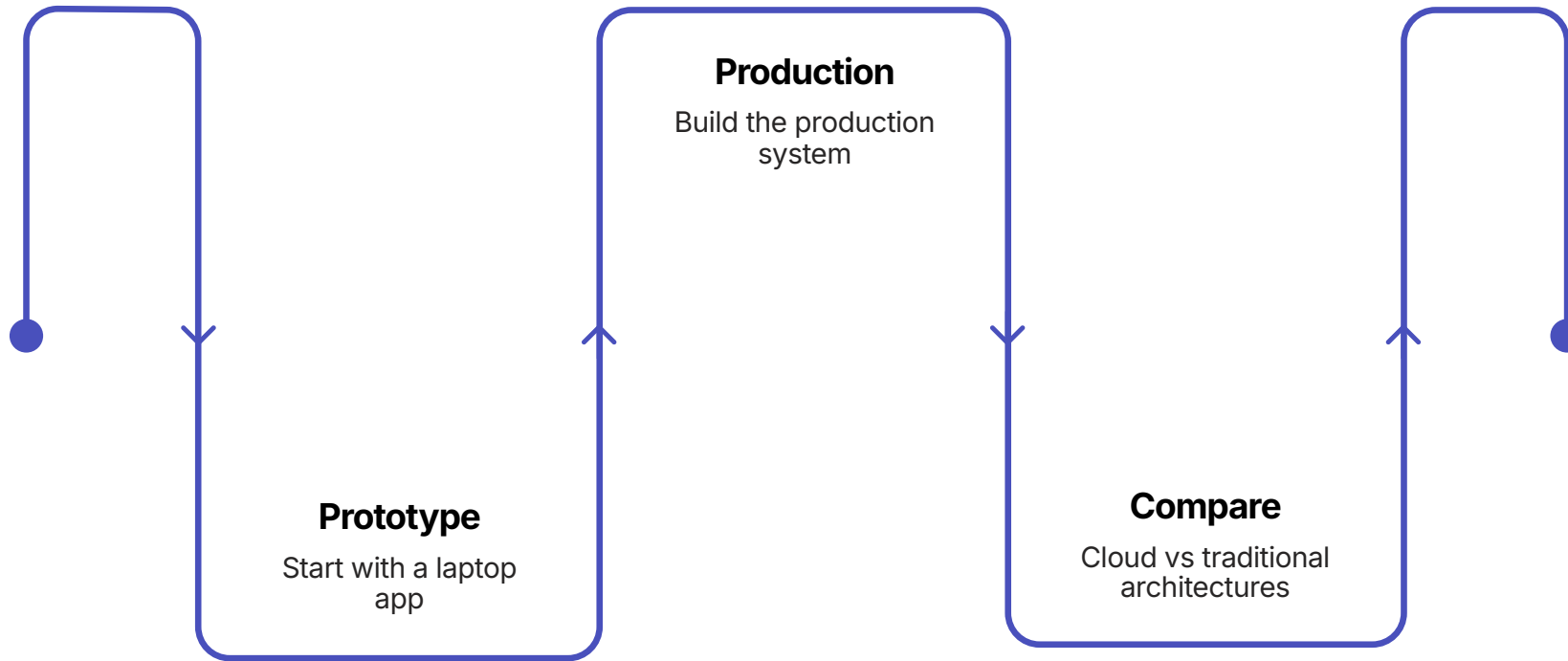


Infrastructure Fundamentals V2

Build a Production System Step by Step

Deck 1 of 2 · Full-Day Training · AM Session



By the end of this deck, you will be able to redraw the ShopEasy production architecture from memory — and explain **why** every component was added.

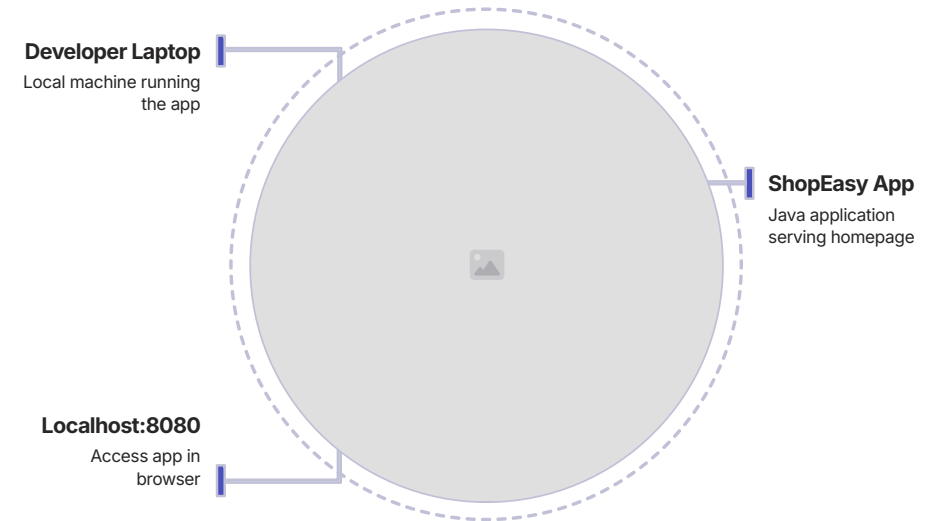
WHERE WE START

You Already Know This

You have a Java application. It compiles. It runs. It works.

You can visit `localhost:8080` in your browser and see the homepage.

✓ This is real. This is working. This is our starting point.



ShopEasy Java App

Browse products · Add to cart · Place order

THE FIRST BIG QUESTION

Is a working app the same as a production system?

Think about it before we turn the page.

THE PROBLEM APPEARS

Meet Priya — Your First Customer

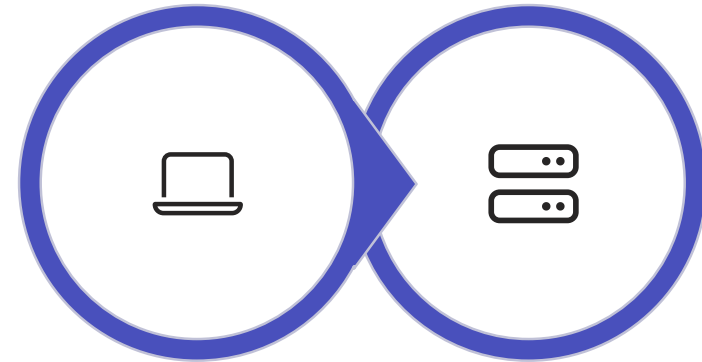
Priya is in another city. She opens her browser and wants to shop on ShopEasy.

Priya types...

localhost:8080

She gets...

Nothing. Connection refused.



Priya (City B)

**Developer
(City A)**

⚠️ Why? Because `localhost` doesn't travel across networks.

KEY CONCEPT

localhost Means *This Machine*

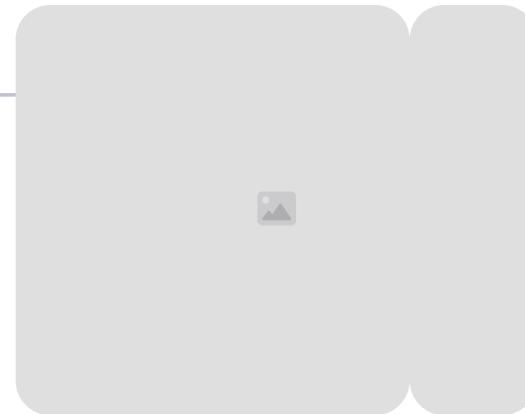
localhost / 127.0.0.1

A special address that loops back to the same machine making the request. No packet leaves the device.

When Priya's browser says localhost, it means **her** machine — not yours. ShopEasy is not there.

Before: Local Request

Browser hits localhost on developer laptop



After: Stays Local

Traffic loops back, never leaves the device or reaches Priya

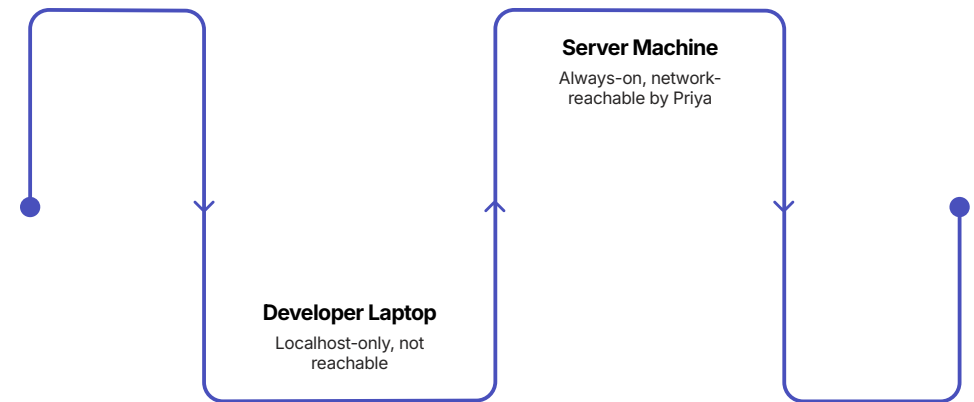
NEXT QUESTION

So We Need a Machine Reachable by Users

Our developer laptop is not reachable from the outside world.

We need a machine that:

- 1 Is always on**
Not sleeping. Not shut down at night.
- 2 Is on a network others can reach**
Has a real address, not just localhost.
- 3 Runs ShopEasy continuously**
Even when no developer is logged in.



That machine has a name: a **SERVER**.

CONCEPT CARD

What Is a Server?

- ❏ A server is not magical hardware. It is a **role** — any machine providing a service to other machines.

The Server Role

Listens for requests.
Processes them. Returns responses. Does this continuously.

Not the Client Role

Your laptop. Priya's phone.
They make requests — they don't serve them.

Your developer laptop *can* be a server. It just isn't a good one for production.



SERVER

Provides a service to network clients



CLIENT

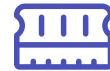
Consumes a service from a server

Open the Box: What's Inside a Server?



CPU

Executes instructions. The brain. Does the actual computing.



RAM

Active working memory. Fast, but temporary. Lost on restart.



Disk / Storage

Persistent data. Survives restarts and power cycles.



NIC

Network Interface Card. Connects the machine to the network.

These four hardware components are in every server — physical or virtual.

CPU: Where Instructions Execute

Every line of your Java code — every if, every loop, every method call — eventually becomes CPU instructions.

CPU speed (GHz) = how many instruction cycles per second.



Fetch instruction → Decode → Execute → Repeat

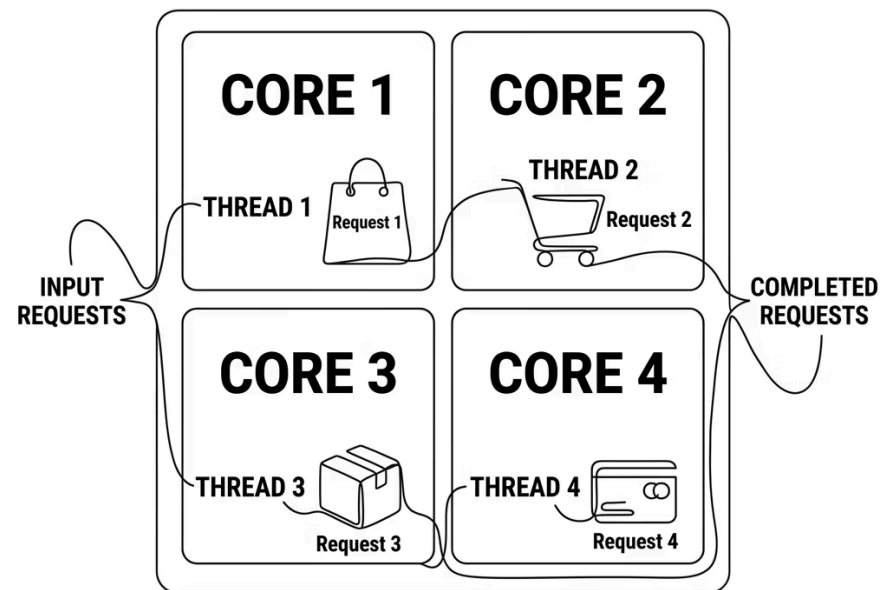
What the CPU Does for ShopEasy

Processes your Java bytecode. Calculates order totals. Renders JSON responses. Checks login credentials.

CPU Cores: Multiple Execution Engines

A single CPU chip can contain multiple **cores** — each an independent execution unit.

ShopEasy with 100 simultaneous users benefits from more cores — the JVM thread pool maps onto cores.



1 Core

One request at a time. Others wait.

4 Cores

Four requests concurrently — each core handles one.

RAM: Active Working Memory

RAM holds what's being worked on *right now*

ShopEasy's JVM heap lives here. Open sessions. Cached objects. In-flight request data.

⚠ RAM is **volatile**. Power off → everything gone. That is not a bug — it is how RAM works.

More RAM = JVM can hold more objects without garbage-collecting constantly = faster responses.

 **RAM**

Fast · Temporary · Expensive per GB

Lost on restart

Disk/Storage: Survives the Restart

Disk is **non-volatile**: magnetic fields or flash cells hold data without power.

Much slower than RAM to read/write — so apps read into RAM, work fast, then write results back to disk.



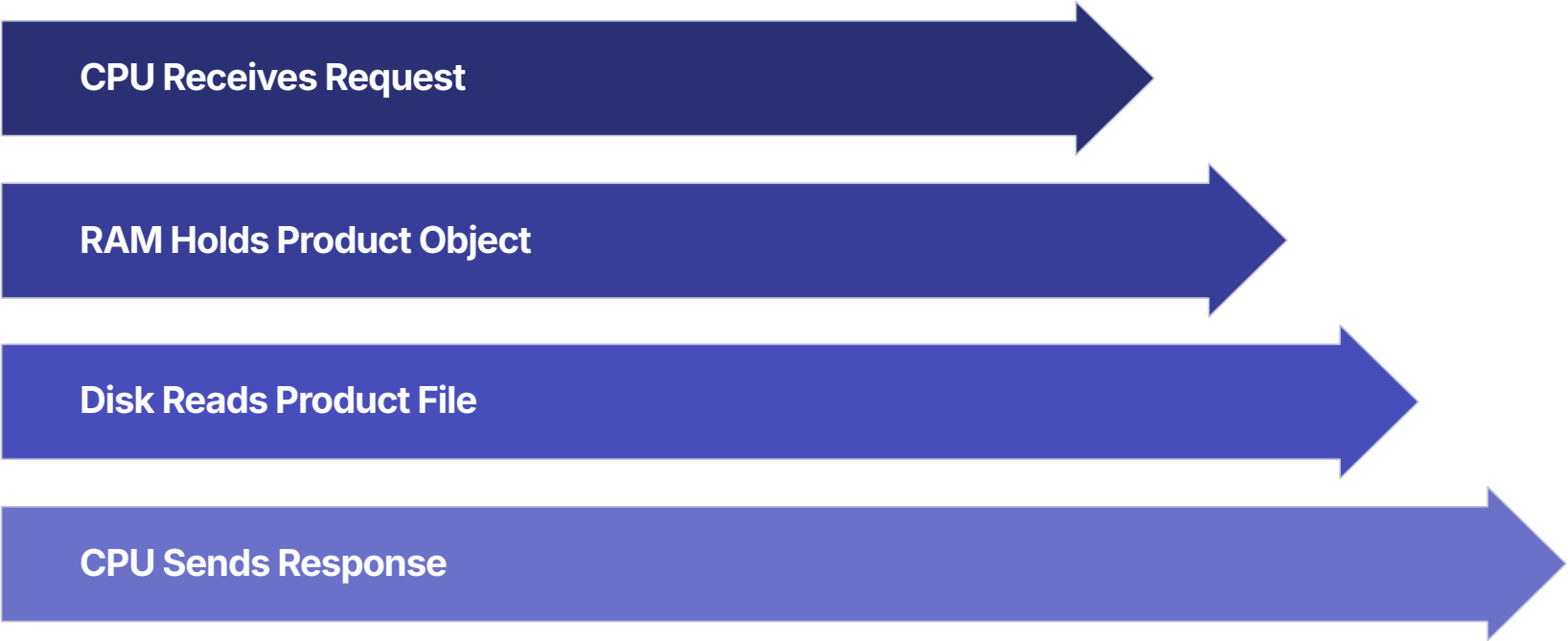
Slow · Persistent · Cheap per GB

Survives restart

Persistent by design

ShopEasy's application JAR lives here. Log files. Configuration. The database files. Anything that must survive a reboot.

CPU vs RAM vs Storage — One ShopEasy Action



Every user action in ShopEasy touches all three components. They are not interchangeable — each has a distinct role.

Component	Speed	Persistent?	ShopEasy Use
CPU	GHz	N/A	Execute Java code
RAM	nanoseconds	No — volatile	JVM heap, sessions
Disk	milliseconds	Yes — survives restart	JAR, logs, DB files

NEXT QUESTION

Hardware Alone Cannot Run ShopEasy

We have CPU, RAM, Disk, NIC. But if we power on a bare server with no software — nothing happens.

Hardware needs a manager. Something that:

- Controls hardware resources
- Lets multiple programs share the CPU and RAM
- Provides file systems, networking, process management

Bare Hardware

CPU · RAM · Disk · NIC

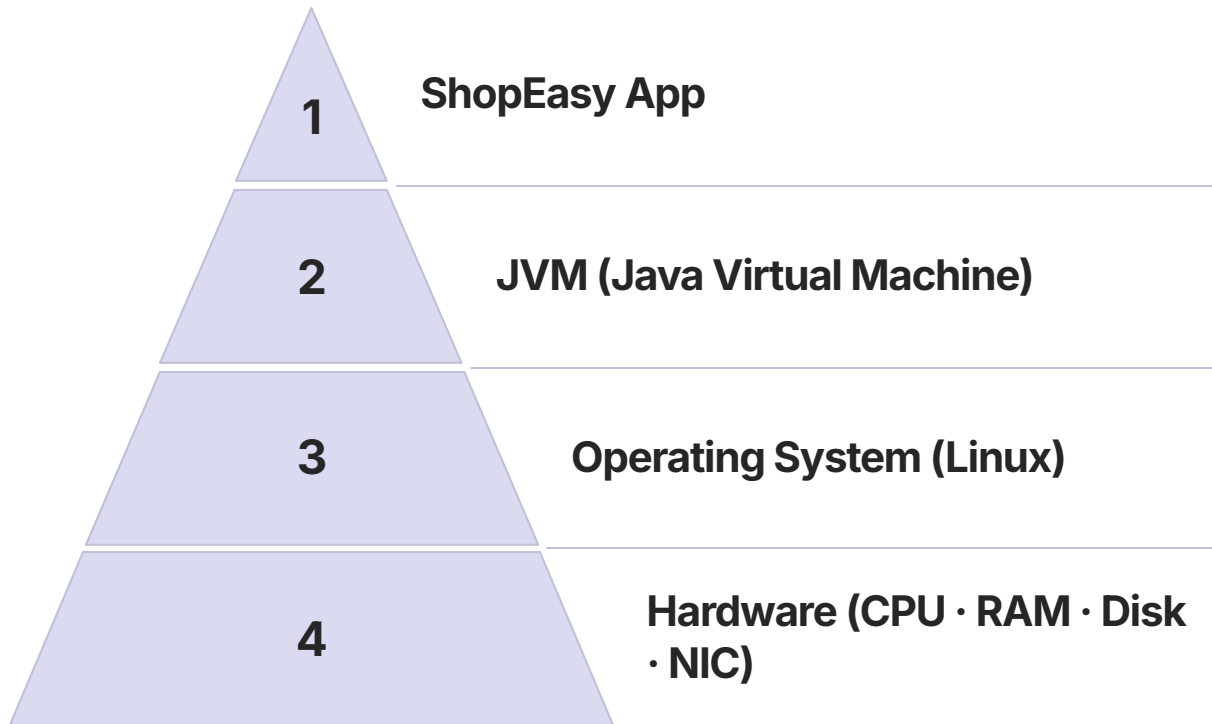
Powered on. Nothing happens.

→ We need software to manage this hardware.

That software is the **Operating System**.

THE STACK APPEARS

The Layered Stack: Hardware → OS → JVM → ShopEasy



OS manages hardware

Allocates CPU time and RAM to processes. Handles files and network.

JVM sits on the OS

Runs your compiled Java bytecode on any OS that has a JVM.

ShopEasy sits on the JVM

Your application code. Pure Java logic.

ShopEasy Running = A Process

When you run `java -jar shopeasy.jar`, the OS creates a new process.
PID might be 4821.

The OS can have many processes running — ShopEasy is just one.

PID 4821

shopeasy.jar

CPU: 12% · RAM: 512MB

PID 1

systemd (OS init)

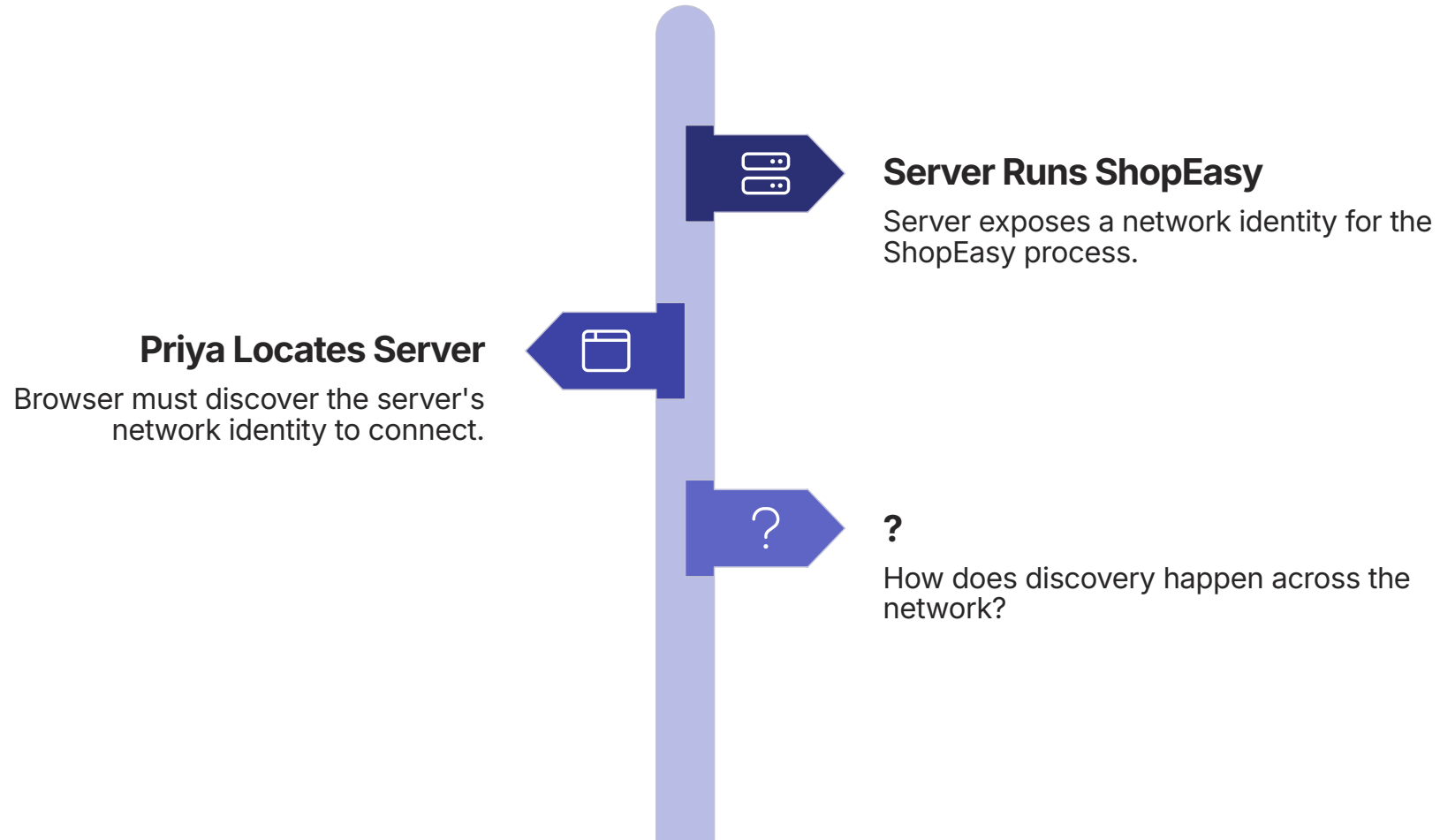
PID 512

nginx (web server)

Process

A running instance of a program. The OS assigns it a **PID** (Process ID), CPU time, and a slice of RAM.

Server is Running ShopEasy. How Does Priya Find It?



The server has an identity on the network. We need to understand that identity before Priya can connect.

IP Address: The Machine's Network Identity

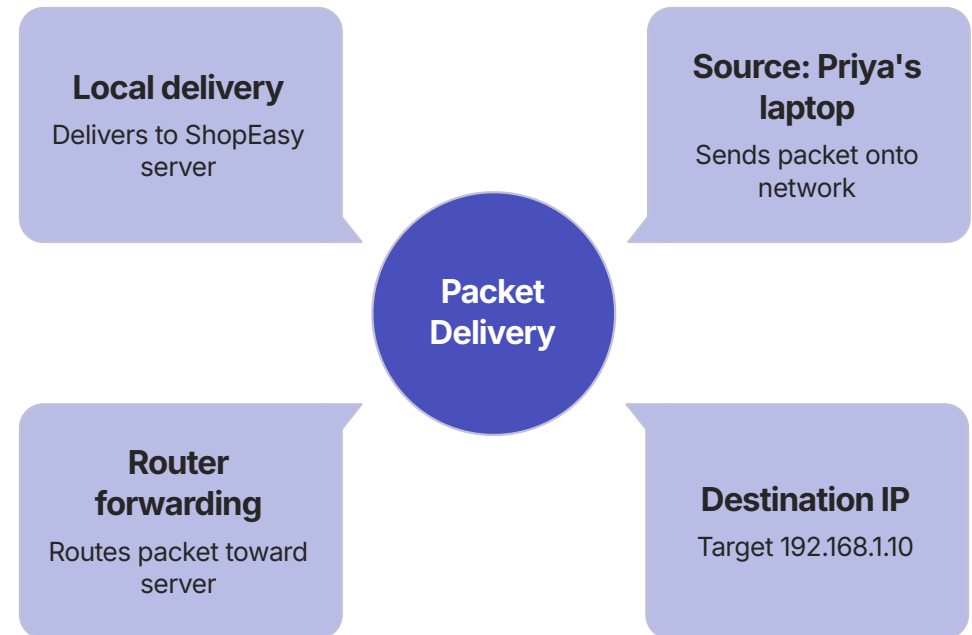
IP Address

A numerical label assigned to a machine on a network.
Routers use it to deliver packets to the right machine.

Like a postal address for a building — tells the network *which machine* to send data to.

Example

192.168.1.10 — private network IP



📌 IP identifies the **machine**, not the application. Multiple programs can run on the same machine with the same IP.

Priya Reached the Machine. Which Program Answers?

The server is running multiple processes: ShopEasy on one, maybe a database on another, maybe SSH for admin access.

IP delivers the packet to the machine. But the OS needs to know **which process** gets it.

Port 8080

ShopEasy

Port 5432

Database

Port 22

SSH (Admin)

Port Number

A number (0–65535) that identifies a specific process/service on a machine. The OS routes incoming packets by port to the correct process.

IP + PORT = ENDPOINT

IP + Port → Application Endpoint

192.168.1.10:8080

192.168.1.10

→ Which machine (the server)

: 8080

→ Which process (ShopEasy)



192.168.x.x is a **private** IP range — used inside a local network. Not routable on the public Internet. We use it here as an example. Public IPs look different and are assigned by your ISP or cloud provider.

Together, IP + Port uniquely identify one process on one machine on a network.

CONCEPT: HTTP/HTTPS

What Language Do Browser and Server Speak?

HTTP — HyperText Transfer Protocol

The agreed format for browser-server conversations. GET a page. POST an order. Server replies with status + body.

HTTPS — HTTP Secure

Same protocol, but the connection is encrypted (TLS). Passwords and card numbers cannot be intercepted in transit.

Port 80

HTTP (unencrypted)

Port 443

HTTPS (encrypted)

ShopEasy speaks HTTP/HTTPS. So does every web browser by default.

FIRST COMPLETE REQUEST

Tracing Priya's First Request End-to-End



The full round trip. Browser → server → response. Priya sees the ShopEasy homepage.

✓ Our system is working! But there is a usability problem with the IP address...

THE HUMAN PROBLEM

**Would you type
192.168.1.10:8080 every
day?**

IPs change. IPs are hard to remember. You can't put an IP address on a business card and expect customers to care.

CONCEPT: DOMAIN NAME

Introduce a Human-Readable Name

Before

192.168.1.10:8080

Hard to remember.
Breaks if IP changes.

After

shopeasy.example

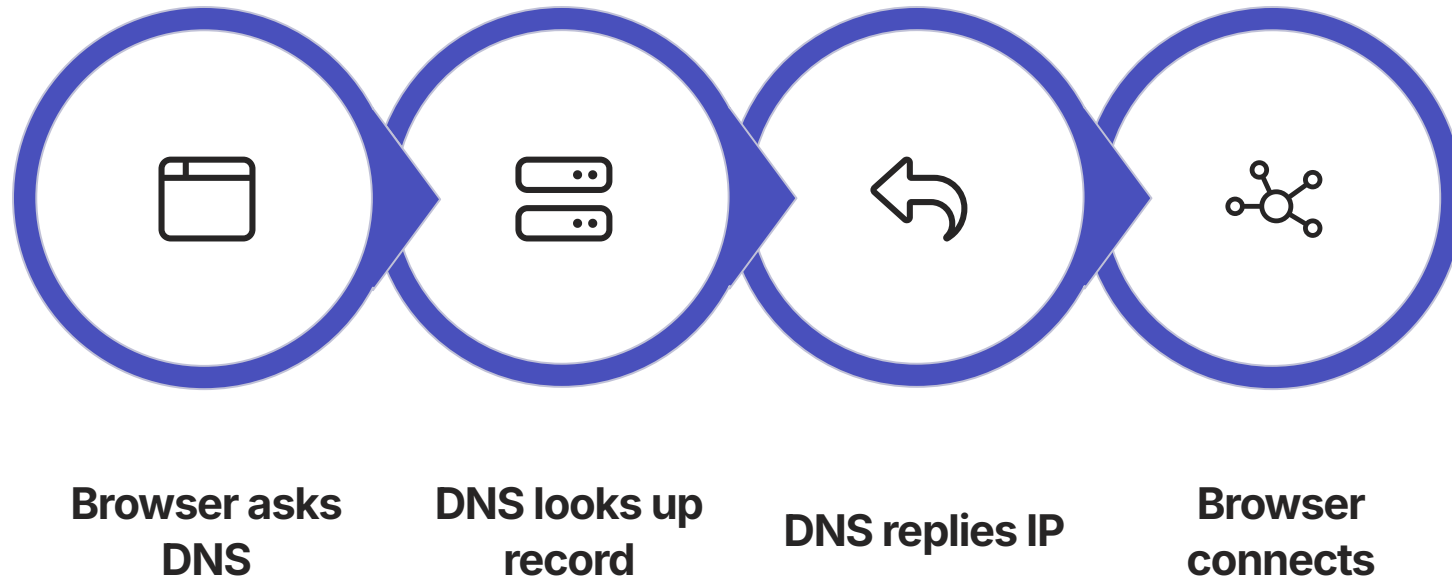
Memorable. Stable.
Brand-friendly.

A **domain name** is a human-readable alias for an IP address.

Something on the network must translate shopeasy.example
→ 192.168.1.10.

That translator is the **DNS**.

DNS: The Phone Book of the Internet



DNS = **Domain Name System**. A distributed directory that maps domain names to IP addresses. The browser caches the answer so it doesn't ask every time.

DNS Record (simplified)

shopeasy.example → A → 192.168.1.10

An 'A record' maps a domain to an IPv4 address.

Full Request: Browser → DNS → IP → ShopEasy



Our system so far: Browser + DNS + Server running ShopEasy.

✓ **Added**

DNS resolution

✓ **Working**

IP + Port endpoint

⚠ **Question**

Is the server safely exposed?

SECURITY QUESTION

Is the Server Directly Exposed to Everyone — Safely?

Right now, anyone who knows the IP can send any kind of traffic to any port.

- Port 22 (SSH admin) — open to the world
- Port 5432 (database) — open to the world
- Malicious bots scan IPs constantly

 Attacker

Scans all ports on your IP

 Bot

Tries default passwords on port 22



Exposing every port to the public internet is not a production system — it is an accident waiting to happen.

Firewall: Traffic Inspector at the Gate

What a Firewall Does

Examines incoming (and outgoing) network packets. Allows or blocks them based on rules.

✓ ALLOW

Port 443 (HTTPS) from anywhere

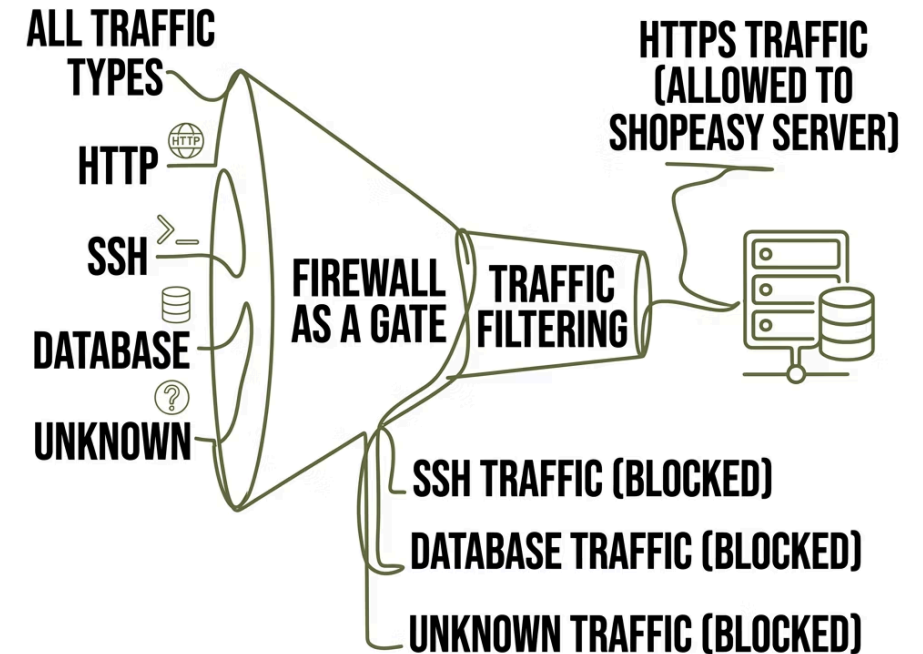
Port 80 (HTTP) from anywhere

✗ BLOCK

Port 22 (SSH) from public IPs

Port 5432 (DB) from public IPs

Everything else — deny



Firewall Joins the Architecture



Our system so far: Browser → DNS → Firewall → ShopEasy Server

 Added

Firewall protection

 Secured

Only HTTPS traffic reaches ShopEasy

 Next

What does the network actually do?

What Is the Network Actually Doing?

When Priya sends a request, it doesn't travel as one big chunk. It's broken into **packets**.

Packets may take different paths and arrive out of order — TCP reassembles them correctly at the destination.

Router

Connects different networks. Decides which path a packet takes toward its destination IP.

Switch

Connects machines within the same local network. Forwards packets by MAC address inside that network.

Packet

A small chunk of data with a header (source IP, destination IP, port) and a payload. The network routes each packet independently.

Router vs Switch — One Clear Comparison

Router

Works between networks

Uses IP addresses

Connects your office to the Internet

Switch

Works within a network

Uses MAC addresses

Connects servers in the same data center rack

- ❏ For sustain engineering: you mostly encounter switches inside data centers and routers at the boundary between networks. You don't need to configure them — but you need to know they exist and what they do.

Our First Real Production Architecture



Every component has a reason. Every arrow represents a real network conversation.



User

Priya's browser initiates the request



DNS

Translates name → IP



Firewall

Allows only port 443



APP Server

Runs ShopEasy process

NEW PROBLEM APPEARS

ShopEasy Needs to Remember Things After Restart

Users place orders. Products have prices, inventory counts, descriptions. User accounts need passwords and addresses.

Where does all of this live?

- In RAM? — Lost when server restarts.
- In a flat file? — Not queryable, not safe for concurrent writes.

After server restart...

All in-memory data: gone.

Priya's order: gone.

Product catalog: gone.



RAM is volatile. ShopEasy needs durable, structured storage.

Introduce the Database: Structured Durable Storage

ShopEasy's database has tables: products, orders, users, inventory.

The database is a **separate process** — often on a separate machine entirely from the APP server.



DB

products table

orders table

users table

What a Database Does

Stores structured data in tables (rows + columns). Handles concurrent reads/writes safely. Lets you query with SQL. Persists data to disk.

Database Joins the Architecture



Our system so far: User → DNS → Firewall → APP → DB

 Added

Database (DB)

 Solves

Durable structured storage

 Next

Where does DB store data physically?

DB → Storage: Where Data Actually Lives

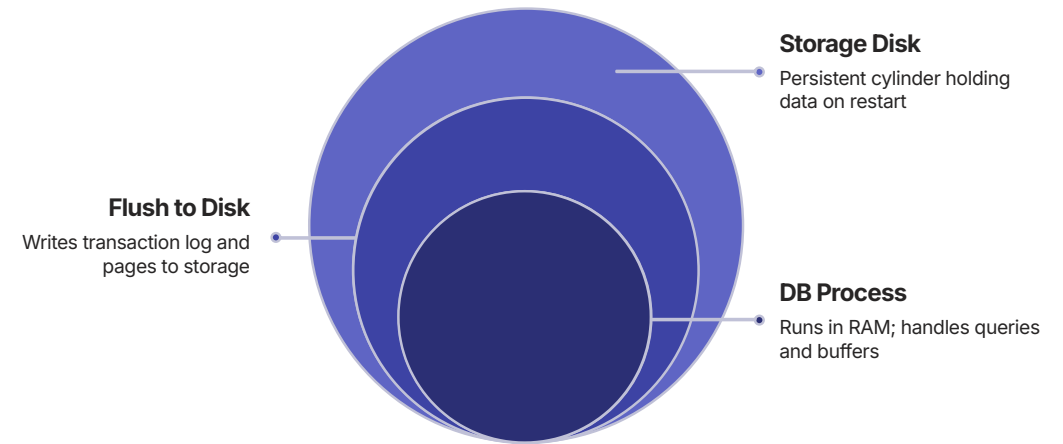
The database process runs in RAM, but writes data to **disk storage** to survive restarts.

DB Process (in RAM)

Active queries. Buffer pool. Transaction log. Index pages cached for speed.

Storage (on Disk)

Data files. WAL (Write-Ahead Log). Backups. This is what persists after a crash.



Where Is That Disk, Exactly?

Local / Direct Storage

Disk physically inside the server. Fast. Simple. But if the server dies, so does the data access.

Network / Shared Storage

Disk is a separate device on the network. Multiple servers can access it. More resilient. Slightly more latency.

- ❏ We don't need to know DAS/NAS/SAN acronyms today. The concept: storage can be inside the box or outside it. Outside storage survives server failures and can be shared. This matters when we talk about resilience in Deck 2.

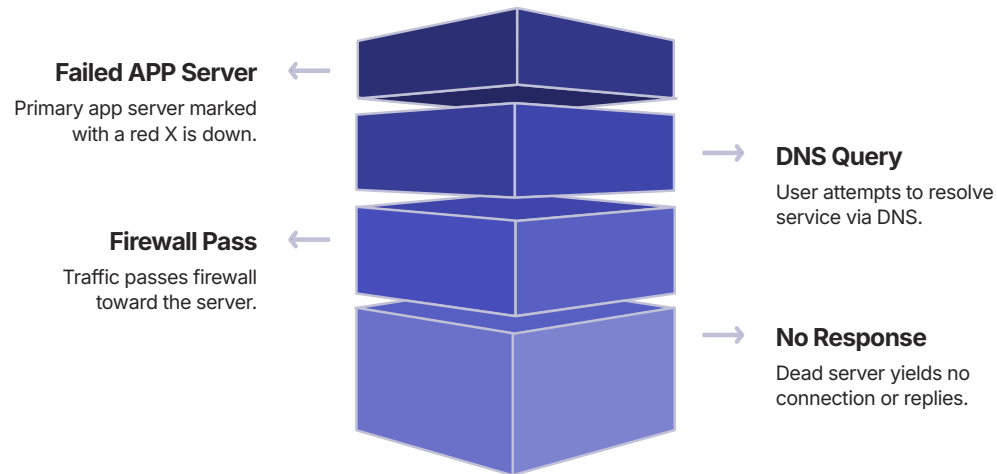
THE SCARY QUESTION

What if the APP server dies?

It's not if. It's when. Hardware fails. Processes crash. OS needs patching. Power cycles happen.

SINGLE POINT OF FAILURE

One APP Server = One Point of Failure



Right now, our single APP server is a SPOF. When it fails, every user — including Priya — gets an error.

□ We'll go deep on resilience in Deck 2. For now: let's just solve the obvious problem.

Single Point of Failure (SPOF)

Any component whose failure causes the entire system to become unavailable.

SIMPLE ANSWER: ADD APP-2

Add a Second Application Server

Both run the same ShopEasy JAR. Both connect to the same DB.

If APP-1 fails, APP-2 can still serve users.



New problem: Priya's browser knows one IP or one domain. Which server does the request go to? How does the traffic split?

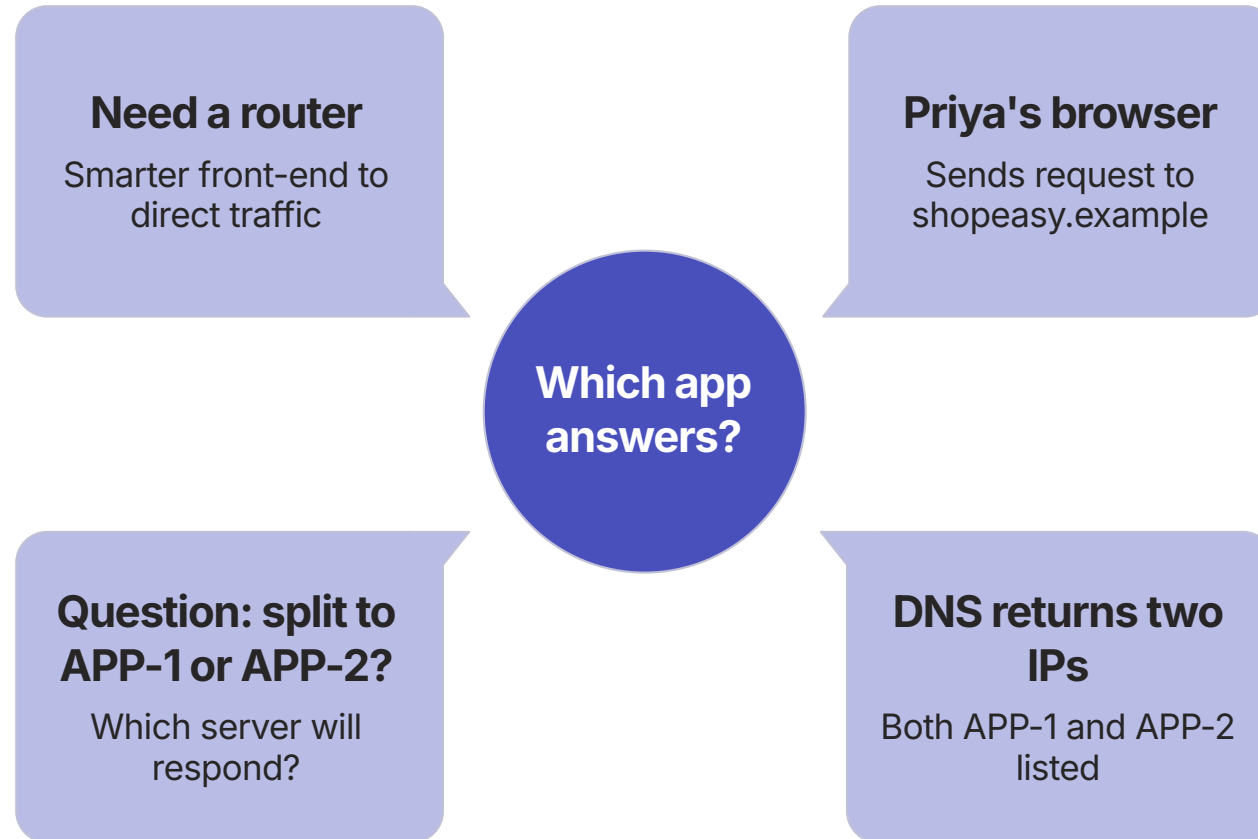
APP-1

ShopEasy process · Port 8080

APP-2

ShopEasy process · Port 8080

Priya Has One Domain. Two Servers. Who Answers?



DNS could give both IPs, but that's fragile — clients cache IPs and wouldn't know if one server died. We need a smarter router in front of our servers.

Load Balancer: One Entry Point, Many Servers

Load Balancer (LB)

Receives all incoming requests at a single address.
Distributes them across available APP servers using a configured algorithm. Monitors server health — stops sending traffic to failed servers.

LB

← One IP in

→ APP-1

→ APP-2

Users only ever know the LB's address. The APP servers are hidden behind it.

Load Balancer in the Architecture



Our system so far: User → DNS → Firewall → LB → APP-1 / APP-2 → DB → Storage

✓ **Added**

Load Balancer

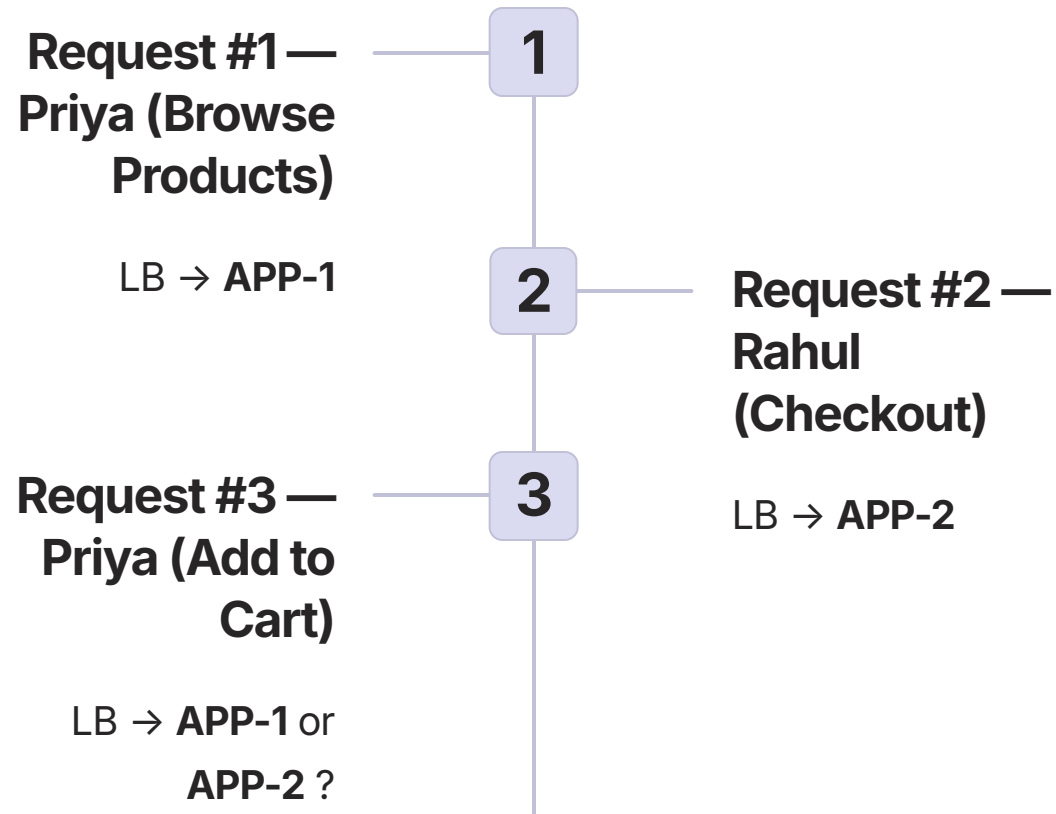
✓ **Solves**

Traffic distribution + basic redundancy

⚠ **New question**

What about user sessions?

Visualizing Request Distribution



Common LB Algorithms

Round Robin: alternate requests 1-2-1-2

Least Connections: send to the server with fewest active requests

IP Hash: same client always hits the same server

- ❏ You don't need to configure algorithms today. Know they exist and have tradeoffs.

Why Application State Creates a Problem

Step 1: Priya logs in

Request goes to **APP-1**.
Session created in APP-1's RAM.
`sessionId=abc123`

1

2

Step 2: Priya adds to cart

Next request goes to **APP-2**. APP-2 has no memory of
`sessionId=abc123`.

Result

APP-2 says: "Who are you? Please log in." Priya is frustrated. Cart is lost.

3



Storing session state in a server's RAM creates a dependency on that exact server. This is the **statefulness problem**.

The Solution: Stateless Application Design

Stateful (problematic)

Session lives in APP server RAM. Next request must go to the same server. All servers are not equal.

Stateless (goal)

Each request carries all context it needs (token/cookie). Server processes it and forgets. Any server can handle any request.

Session data that must persist goes in a **shared store** — the database, or a dedicated session cache — not in server RAM.

- ✔ Stateless apps + a load balancer = true horizontal scalability. Any server, any request.

GROWTH PROBLEM

ShopEasy Goes Viral. Traffic Spikes 10x.

Normal Day

100 requests/minute ·
CPU 20% · RAM 30%

Flash Sale

1,000 requests/minute ·
CPU 95% · RAM 85%

Response time: 8
seconds. Users
abandon cart.

The servers are the bottleneck. We have two options for adding capacity.

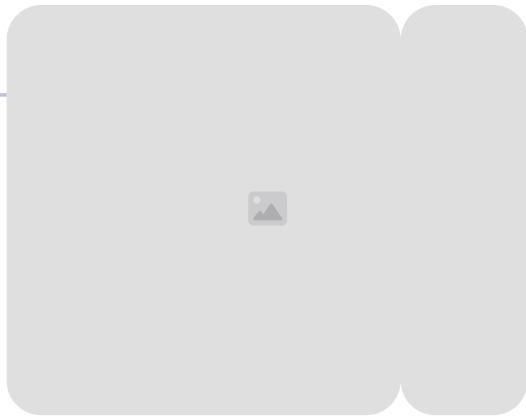
Understanding the tradeoffs will shape every infrastructure decision you make.

SCALE UP

Scale Up: Give the Server More Power

Before: APP-1

4 CPU, 8GB RAM
small server



After: APP-1

16 CPU, 64GB RAM
bigger server

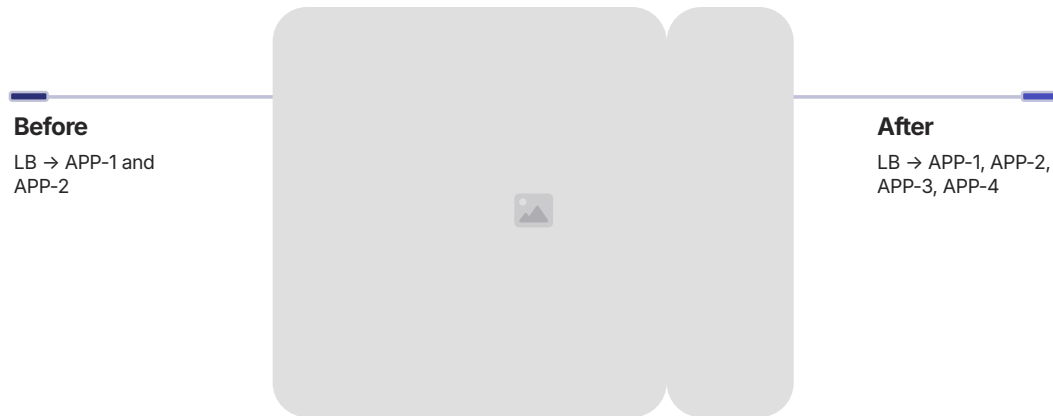
-  Simple — no code changes needed
-  Hardware has a ceiling — you can't add CPUs forever
-  Expensive beyond a point
-  Still one machine — still a SPOF

Vertical Scaling (Scale Up)

Replace the machine with a more powerful one. More CPU cores, more RAM, faster disk.

SCALE OUT

Scale Out: Add More Servers



-  No ceiling — add as many as needed
-  Removes SPOF — any server can fail without outage
-  Requires stateless app design
-  More machines to manage

Horizontal Scaling (Scale Out)

Add more servers of the same size. Load balancer distributes traffic across all of them.

Side by Side: When to Use Each

Factor	Scale Up ↑	Scale Out →
Approach	Bigger single machine	More machines of same size
Code changes?	None	App must be stateless
Hardware ceiling	Yes — limited	No — add indefinitely
Fault tolerance	Still a SPOF	Other servers absorb traffic
Best for	Databases, quick fixes	Web/app servers, sustained growth

📌 In practice, you often do both: scale up the DB, scale out the app tier.

PERFORMANCE PROBLEM

Priya Browses Products. So Does Everyone Else.

The ShopEasy product catalog has 10,000 items. Every page load queries the database.

The database becomes the bottleneck. Products rarely change. Why hit the DB every time?



Reading the same data from disk over and over is wasteful. We need a faster layer for repeated reads.

1,000 users browsing = 1,000 identical DB queries

```
SELECT * FROM products WHERE category='shoes'
```

Same query. Same result. 1,000 times per minute.

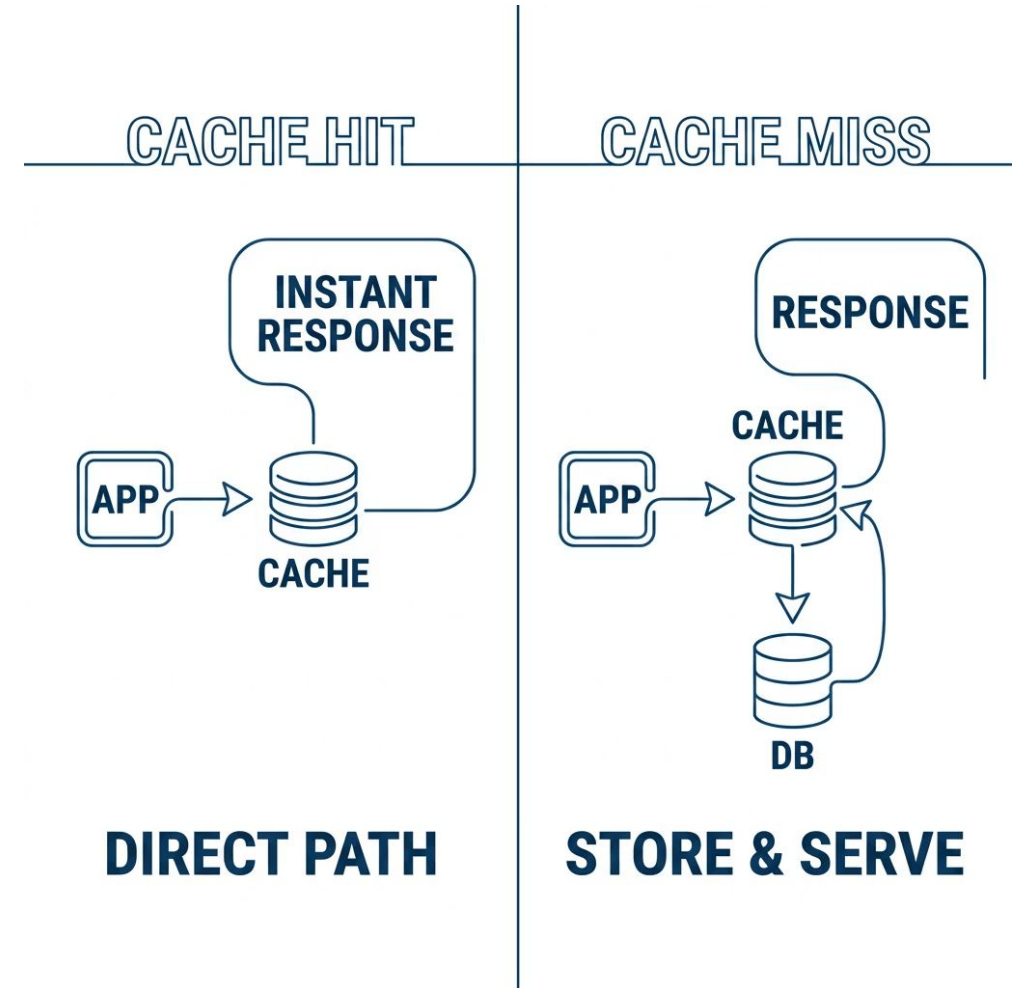
Cache: Store the Answer, Skip the Work

Cache Hit

APP asks CACHE for product list.
CACHE has it. Returns instantly from
RAM. No DB query needed.

Cache Miss

APP asks CACHE for product list.
CACHE doesn't have it. APP queries
DB. DB returns result. APP stores it in
CACHE for next time.



Cache Joins the Architecture



Our system so far: User → DNS → Firewall → LB → APP cluster → CACHE → DB → Storage

✓ **Added**

CACHE (e.g., Redis)

✓ **Solves**

Repeated read performance

⚠ **Next**

How do we know if it's all working?

How Do Engineers Know the System Is Healthy?

Monitoring

Continuously measures system metrics. CPU%, RAM%, response time, error rate, DB connections, cache hit rate. Alerts when thresholds are crossed.

Logging

Records events as they happen. Every request, every error, every exception. Essential for diagnosing what went wrong and when.



Monitoring

Real-time health dashboards



Logging

Historical event trail



Alerting

Notify on-call engineer

☐ Monitoring is infrastructure too — not an optional add-on. Without it, you're flying blind.

The Complete ShopEasy Production Architecture



Every component was added to solve a real problem. You can now explain why each one is there.

DNS

Name → IP

Firewall

Traffic control

LB

Distribute + redundancy

APP cluster

ShopEasy processes

CACHE

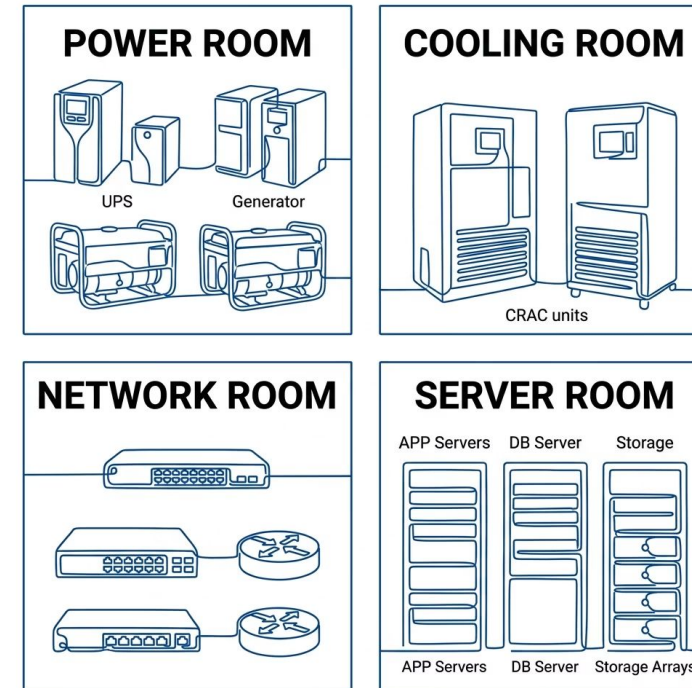
Fast repeated reads

DB + Storage

Durable data

Where Are All These Machines Physically?

Our architecture is a set of logical boxes. Physically, those boxes are machines sitting in a **data center**.



What a Data Center Provides

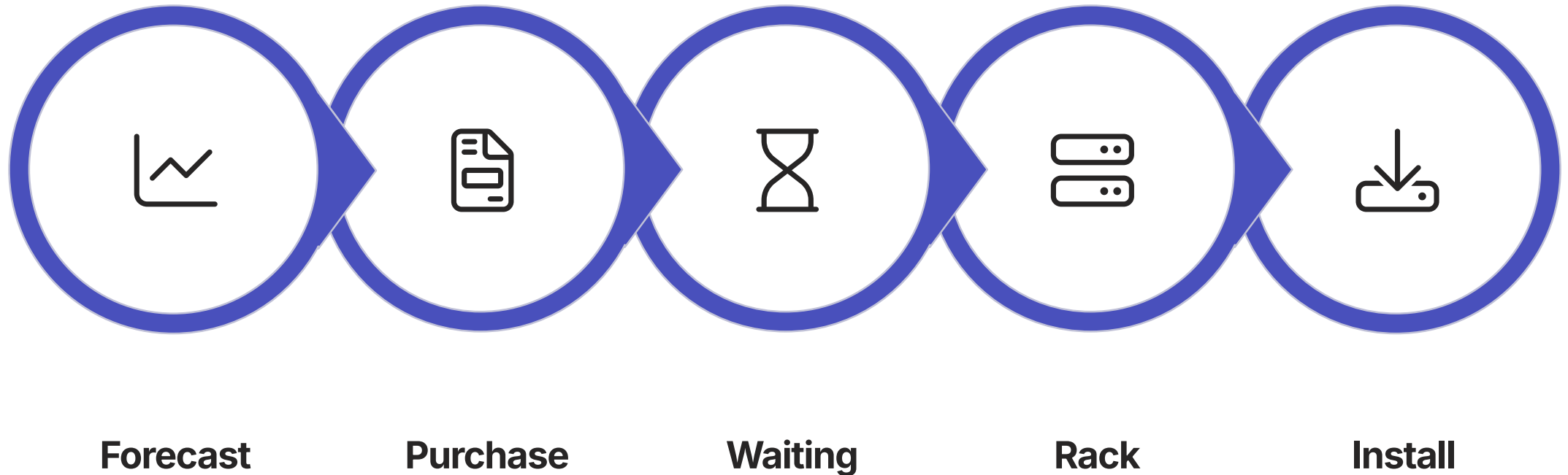
Power: Redundant power feeds + UPS + generators

Cooling: Servers generate heat — precision AC keeps them alive

Networking: Switches, routers, uplinks to the Internet

Physical security: Locked cages, access control, cameras

The On-Premises Hardware Lifecycle



From capacity decision to production: often **8–16 weeks**. The business doesn't wait 16 weeks.

THE CORE PROBLEM WITH TRADITIONAL IT

Demand Changes. Hardware Procurement Doesn't.

Under-Provisioned

Bought too little. ShopEasy flash sale. Servers at 100%.
Users get errors. Revenue lost.

Over-Provisioned

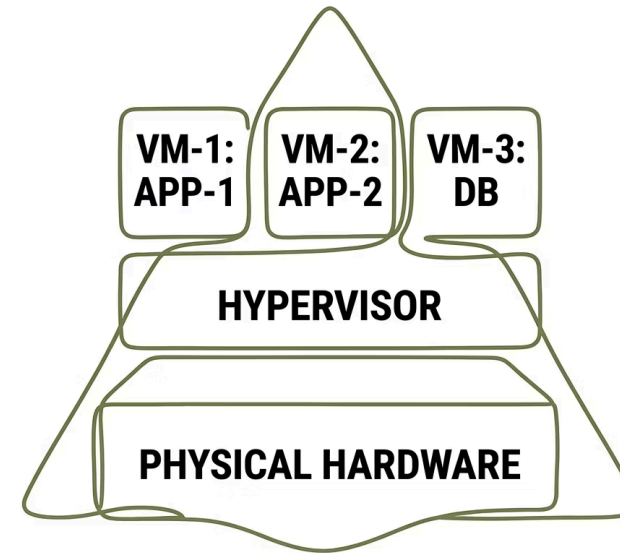
Bought too much. Normal traffic. Servers at 10%.
Expensive hardware sitting idle. Money wasted.

 Physical hardware forces you to choose: waste money or risk capacity. There is rarely a perfect fit.

Virtualization: One Physical Machine, Many Virtual Ones

- ☒ Better hardware utilization — one physical server, multiple workloads
- ☒ Faster to provision than buying physical servers
- ☐ ⚠ Still your hardware — you still bought and own it

☐ Virtualization ≠ Cloud. Running VMs on your own hardware is still on-premises — just more efficient.



Hypervisor

Software layer that sits between hardware and the OS. Divides physical CPU/RAM/Disk into multiple isolated **Virtual Machines (VMs)**. Each VM looks like a real machine to its OS.

The Same ShopEasy Architecture — Now on Cloud

Traditional / On-Premises

You own the hardware

Buy → Rack → Configure → Operate → Replace

Capacity fixed at purchase time

You manage: hardware, OS, network, cooling, power

Cloud / On-Demand

Provider owns the hardware

Request → Provision in minutes → Use → Release

Capacity adjusts with demand

You manage: configuration, app, data

IaaS

You get VMs. You manage OS up.

PaaS

You get a platform. Deploy app code.

SaaS

You get a service. Just use it.

Public Cloud

Shared infra (AWS/GCP/Azure)

Private Cloud

Dedicated infra, cloud model

Hybrid

Mix of both, bridged